Anieesh Saravanan - Algorithms HW 1

Problem 1:

1. z is the deepest ancestor that becomes unbalanced after insertion. Let height(z) then represent its height before the insertion takes place.
Before insertion, z has a BF of -1 or +1, because the insertion can only increase a child's subtree by 1 to cause the BF to reach -2 or +2.
Its child subtrees have heights (height(z)-1) and (height(z)-2) as a result.

Assume BF(z) = +2 after insertion. This yields two distinct scenarios:

Scenario 1: LL - Left Left

The insertion increases the height of the left subtree of z's left child. A right rotation at z brings the balance back.
After the rotation, the original z node has two child subtrees of height (height(z)-2), causing its height to be:

$$1 + \max(\text{height}(z)-2, \text{height}(z)-2) = \text{height}(z)-1$$

The node that got promoted in the rotation has two child subtrees of height (height(z)-1), so its height is:

$$1 + \max(\text{height}(z)-1, \text{height}(z)-1) = \text{height}(z)$$

The subtree, after rebalancing, has the same height it had before the insertion took place.

Scenario 2: LR - Left Right

The insertion increases the height of the right subtree of z's left child. A left rotation on the left child and a right rotation at z brings the balance back.
After these 2 rotations, the two children of the new root both have height (height(z)-1).
Thus, the rebalanced subtree has heights:

$$1 + \max(\text{height}(z)-1, \text{height}(z)-1) = \text{height}(z)$$

The subtree, after rebalancing, has the same height it had before the insertion took place.

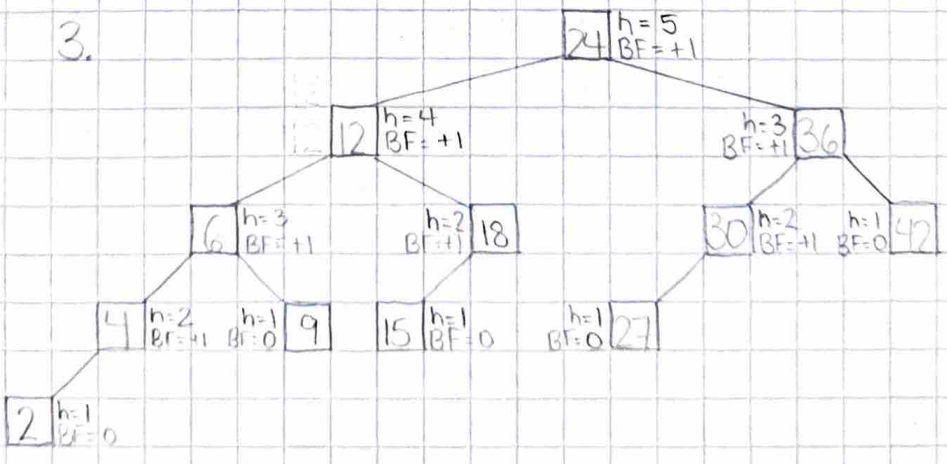
1. By symmetry, the Right Right and Right Left scenarios (discussed) will cause the same result.

In all possibilities, a single/double rotation brings the unbalanced subtree back to height height(z). The height is constant, so the BF of all ancestors above z remains the same. This proves AVL insertion only needs one restructuring, as no more rotations are necessary.

2. Even after rebalancing, deletion can reduce the height of a subtree by 1.
Let height(z) represent the height of the subtree with root z before deletion. Imagine its left and right subtrees have (height(z)-1) and (height(h)-2). Then BF(z) = +1. If deletion brings the right subtree height to (height(z)-3), then BF(z) = +2. A right rotation would bring the balance back, but the restructured subtree would have height (height(z)-1). The height decrease propagates to the parent of z. If the parent was already heavy on the other side, its BF could become +2 or -2, causing another rotation. This continues up through ancestors, potentially reaching the root.

assuming BF(left) = +1
↳

3.



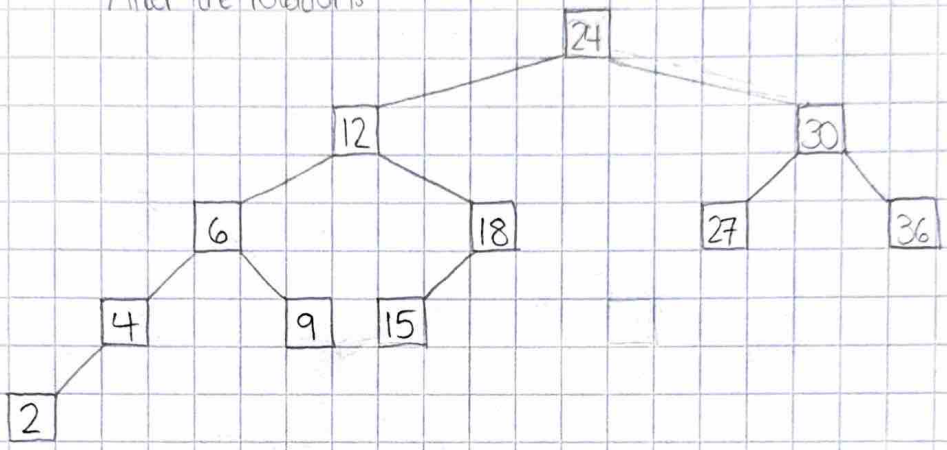
Deletion of the leaf with key 42 would trigger a rebalancing rotation at every level along its ancestor path to the root.

4. Step 1: delete 42

BF(36) = 2 - 0 = +2 ← imbalanced node 36

BF(30) = +1, LL imbalance, rotate right at node 36

After the rotations:

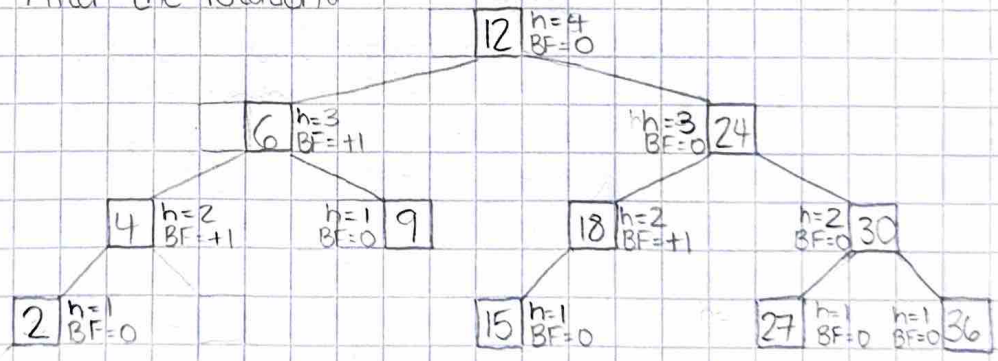


Step 2: Rebalance node 24

BF(24) = 4 - 2 = 2 ← imbalanced node 24

BF(12) = 3 - 2 = +1, LL imbalance, rotate right at node 24

After the rotations:

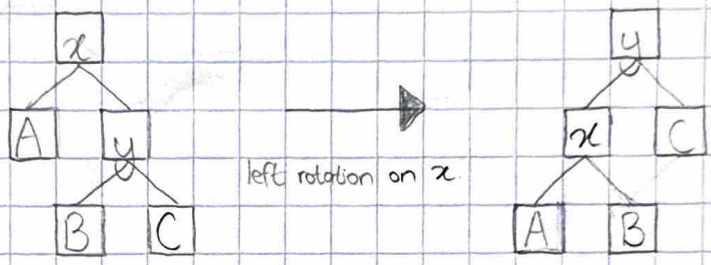


Deleting leaf 42 necessitated two right rotations (at nodes 36 and 24) to restore balance.

Problem 28

1. $x.size = 1 + x.left.size + x.right.size$
The size of a subtree is dependent on just the node and its immediate children. Thus, maintaining subtree sizes does not increase the asymptotic $O(\log n)$ update time of an AVL tree.

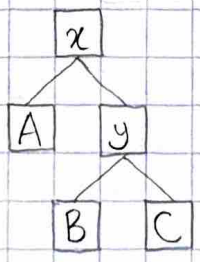
2. $y = x.right$:



The subtrees A, B, and C are completely unchanged.

After rotation:
$x.size = 1 + size(A) + size(B)$
$y.size = 1 + x.size + size(C) = 2 + size(A) + size(B) + size(C)$

3. Take this tree:



Let $size(A) = size(B) = size(C) = s$

After a left rotation on x, the depth of nodes in subtree A increases by 1, while nodes in C decrease by 1.

The depths of node x and y are also changed.

Thus, at least $2s+2$ depth values must be updated following the rotation. Since the tree has $n = 3s+2$ nodes, $2s+2 = \Omega(n)$.

A single rotation can require $\Omega(n)$ updates for depth. Storing explicit absolute depth cannot be maintained in $O(1)$ time per rotation.

4. Just do a BST search that counts the number of edges traversed on the fly instead of storing each of the absolute depths. The search only goes down a single path from root to the node being queried, and since an AVL tree has height $O(\log n)$, it takes $O(\log n)$ worst-case time without having to store absolute depth fields.

Problem 3.

1. True Sleator-Tarjan Zig-Zig rotations:
   Each zig-zig step does two right rotations. First, the parent is rotated above the grandparent, then key 1 is rotated above the parent. Key 1 is effectively moved upward by two levels and the original chain is split into shorter branches. After key 1 reaches the root, even keys are in a descending linear chain on its right side with odd keys as right "offshoot" branches. The height is the longest path, involving the node 1 root, even numbered keys (which there are $\lfloor n/2 \rfloor$ of), and the last "offshoot" node 3. The height is $\lfloor n/2 \rfloor + 2$

   This assumes there are more than 2 nodes →

2. Naive repeated single rotations:
   Key 1 is rotated above its parent one level at a time until it eventually reaches the root. The keys remain in a linear chain, with 1 having right child n, having left child n-1, ..., 3, 2 as left children in succession. The height of this tree is n.

Only the actual Zig-Zig rotations involve halving height of the tree.

2. Using the naive repeated single rotations, getting key 1 necessitates $n-1$ rotations. Key 1 is the root, but other keys are still arranged in a linear chain under its right child. Getting key 2 also necessitates $n-1$ rotations. Key 2 becomes the root but the bigger keys are still in a linear arrangement. All other accesses (following 2) needs 1 less rotation each time:

$$T(n) = (n-1) + (n-1) + (n-2) + \cdots + 1$$
$$= (n-1) + [n(n-1)/2] = \underline{\Theta(n^2)}$$

With n accesses, the amortized cost per operation:

$$T(n)/n = \underline{\Theta(n)}$$

3. A Zig-Zig step brings accessed nodes two levels nearer to the root node by rotating the parent above the grandparent before rotating the accessed node. Rotations also spread ancestors into different branches, such that $\approx 1/2$ the original chain stays on the longest path. The naive repeated single rotations move the accessed node towards the root, one level per step. This process does not distribute the linear chain of the ancestors, so the tree keeps its linear height.

Why halved? As stated before, the tree is actually restructured so even nodes stay in a descending chain while the odd nodes become side branches.

Problem 4:

1. A Zig-Zag step only changes the ranks of nodes x, p, and g.
   Actual cost is 2 rotations:
   $$\Delta\Phi = r'(x) + r'(p) + r'(g) - r(x) - r(p) - r(g)$$

2. After the rotations take place, x occupies g's former position as stated and its subtree has the same number of nodes as g's original subtree:
   $$s'(x) = s(g)$$
   $$r'(x) = r(g)$$
   Since $c = 2$, substitute in the formula:
   $$\hat{c} = 2 + r(g) + r'(p) + r'(g) - r(x) - r(p) - r(g)$$
   $$= 2 + r'(p) + r'(g) - r(x) - r(p)$$

3. x is p's child before rotation, so $s(x) \leq s(p)$.
   Thus, $r(x) \leq r(p)$. So, $-r(p) \leq -r(x)$. Using this:
   $$\hat{c} \leq 2 + r'(p) + r'(g) - 2r(x)$$

4. After the Zig-Zag rotations, x is the parent of p and g. Since p and g are two separate children of x and every node in a tree has a single parent, their subtrees cannot share any nodes (and are disjoint).
   The subtree with root x contains x and the two disjoint subtrees. Thus:
   $$s'(x) = 1 + s'(p) + s'(g)$$
   $$s'(p) + s'(g) \leq s'(x)$$
   Let $s_1 = s'(p)$ and $s_2 = s'(g)$. $(s_1 - s_2)^2$ is guaranteed to be greater than or equal to 0 ($r^2 \geq 0$, $r \in \mathbb{R}$).
   $$s_1^2 - 2s_1s_2 + s_2^2 \geq 0$$
   $$s_1^2 + 2s_1s_2 + s_2^2 \geq 4s_1s_2 \quad \text{(add } 4s_1s_2 \text{ to both sides)}$$
   $$(s_1 + s_2)^2 \geq 4s_1s_2$$
   $$s_1s_2 \leq ((s_1+s_2)/2)^2 \rightarrow \log_2(s_1) + \log_2(s_2) \leq 2\log_2((s_1+s_2)/2)$$
   Thus, $\log_2(s_1) + \log_2(s_2) \leq 2\log_2(s'(x)) - 2$
   Substitute using $s_1 = s'(p)$, $s_2 = s'(g)$, $r(x) = \log_2(s(x))$
   $$r'(p) + r'(g) \leq 2r'(x) - 2$$
   $$\therefore 2 + r'(p) + r'(g) \leq 2r'(x)$$

5. Part 3: $\hat{c} \leq 2 + r'(p) + r'(g) - 2r(x)$
Part 4: $2 + r'(p) + r'(g) \leq 2r'(x)$
$\hat{c} \leq 2r'(x) - 2r(x)$
$\hat{c} \leq 2(r'(x) - r(x))$
The subtree of x grows after the rotations take place. Thus, $s'(x) \geq s(x) \longrightarrow r'(x) \geq r(x)$
Therefore:
$$2(r'(x) - r(x)) \leq 3(r'(x) - r(x))$$
$$\hat{c}_{2g-2og} \leq 2(r'(x) - r(x)) \leq 3(r'(x) - r(x))$$

Problem 5:

1. In order to fit in a 4096 byte page:
$$\underbrace{32(M-1)}_{\text{keys}} + \underbrace{8(M-1)}_{\text{record pointers}} + \underbrace{8M}_{\text{child pointers}} \leq 4096$$
$48M - 40 \leq 4096$
$M \leq 4136/48 \approx 86.1$
The maximum B-tree order is $M = 86$

$$\begin{array}{r} 86.1 \\ 48\overline{)4136} \\ -384 \\ \hline 296 \\ -288 \\ \hline 80 \\ -48 \\ \hline 32 \end{array}$$

2. $M = 86$, the minimum # of children in internal nodes that are not the root is $t = \lceil 86/2 \rceil = 43$. The root has at least 1 key and 2 children. At depth $d \geq 1$, there are $\geq 2t^{(d-1)}$ nodes with $\geq t-1$ keys each. Thus, using h, the minimum # of keys is:
$$1 + 2(t-1)(1 + t + \cdots + t^{h-1})$$
$$1 + 2(t-1)\frac{t^h - 1}{t-1} = 2t^h - 1$$
With 10^7 keys and $t = 43$:
$$2(43^h) - 1 \leq 10^7$$
$$43^h \leq (10^7 + 1)/2$$
$$h \leq \log_{43}\left(\frac{10^7+1}{2}\right) \approx 4.101$$
The maximum B-Tree height is 4 edges.

3. A binary tree with height h has a maximum of $2^{h+1}-1$ nodes.

$$2^{h+1}-1 \geq 10^7$$
$$h \geq \log_2(10^7+1)-1$$
$$h_{min} = \lceil \log_2(10^7+1) \rceil - 1 = 23 \text{ edges}$$

From Problem 1, $N(h) = 1 + N(h-1) + N(h-2)$
We proved $N(h) = F_{h+2} - 1$ in class, with
$F_k \approx \frac{\varphi^k}{\sqrt{5}}$, $\varphi = 1.61803$

So $N(h) \approx \frac{\varphi^{h+2}}{\sqrt{5}} - 1$

$$\frac{\varphi^{h+2}}{\sqrt{5}} - 1 \leq 10^7 \longrightarrow \varphi^{h+2} \leq (10^7+1)\sqrt{5}$$
$$h+2 \log(\varphi) \leq \log((10^7+1)\sqrt{5})$$
$$h \leq \frac{\log((10^7+1)\sqrt{5})}{\log(\varphi)} - 2 \approx 33.17$$
$$h_{max} = 32 \text{ edges}$$

4. The minimally filled B-Tree needs a maximum of 4 disk I/O operations, while the AVL tree needs a maximum of 32 disk I/O operations. On the enterprise hard drive, the structures have a maximum lookup time of 40ms, 320ms respectively. Multiway trees are essential because the branching factor minimizes the tree height, which reduces the number of disk accesses necessary for a lookup.

Problem 6:

1. Method: range_sum (T, low, high)
   Inputs: An augmented BST root T where each node x has fields $x.key$, $x.left$, $x.right$, $x.sum$, and interval [low, high] with low ≤ high

```
x := T
while x ≠ NULL and (x.key < low or x.key > high):
    if x.key < low:
        x := x.right
    else:
        x := x.left
if x = NULL:
    return 0
s := x.key, n := x.left
while n ≠ NULL:
    if n.key < low:
        n := n.right
    else:
        s := s + n.key
        if n.right ≠ NULL:
            s := s + n.right.sum
        n := n.left
n := x.right
while n ≠ NULL:
    if n.key > high:
        n := n.left
    else:
        s := s + n.key
        if n.left ≠ NULL:
            s := s + n.left.sum
        n := n.right
return s
```

   Output: The exact sum of all keys in T satisfying low ≤ key ≤ high

2. The first loop searches for a node x satisfying $low \leq x.key \leq high$. When a node has a key not in the desired range, the algorithm follows a subtree that could contain keys in the range (by exploiting BST ordering). If the algorithm gets to NULL, there are no keys in the range, and it's right to return 0.
If x is successfully found, it's added to the running sum S. Every key in the $x.left$ subtree is guaranteed to be below high., so the second loop only does bound checks on the lower bound as it iterates. If $n.key \leq low$, it passes over n's left subtree and n because all keys contained are not in the range. Otherwise, $n.key$ and $n.right.sum$ is added, as the entire right subtree is guaranteed to be in the interval (below high and not yet below low). It continues left and performs the conditional at each level.

The third while loop is symmetric to the second while loop on $x.right$, by checking the upper bound and adding sums from the left subtree

Every key within the interval is added. Only subtrees that cannot contain keys in the interval are passed over. Node x and its subtrees are disjoint by nature, and loops only add subtrees that are disjoint from ones it reaches after. No keys are left out, and the disjointedness ensures the sum is accurate.

Thus, the algorithm is correct.

3. The loops pick a single path to go down and does $O(1)$ work per node.

$T(h) = T(h-1) + O(1)$
$T(0) = O(1)$

Solve: $T(h) = O(h)$
↳ the algorithm goes down a maximum of 3 paths, and the total run time remains at $O(h)$ as a result.
AVL trees: $h = O(\log n)$

Therefore, $T(n) = O(\log n)$. This is completely independent of the number of keys in the range, as subtrees are added in $O(1)$ time, so the requirements are satisfied.